

Apache Hadoop

From Wikipedia, the free encyclopedia

Apache Hadoop is an open-source software framework written in Java for distributed storage and distributed processing of very large data sets on computer clusters built from commodity hardware. All the modules in Hadoop are designed with a fundamental assumption that hardware failures (of individual machines, or racks of machines) are commonplace and thus should be automatically handled in software by the framework.^[3]

The core of Apache Hadoop consists of a storage part (Hadoop Distributed File System (HDFS)) and a processing part (MapReduce). Hadoop splits files into large blocks and distributes them amongst the nodes in the cluster. To process the data, Hadoop MapReduce transfers packaged code for nodes to process in parallel, based on the data each node needs to process. This approach takes advantage of data locality^[4]—nodes manipulating the data that they have on hand—to allow the data to be processed faster and more efficiently than it would be in a more conventional supercomputer architecture that relies on a parallel file system where computation and data are connected via high-speed networking.^[5]

The base Apache Hadoop framework is composed of the following modules:

- *Hadoop Common* – contains libraries and utilities needed by other Hadoop modules;
- *Hadoop Distributed File System (HDFS)* – a distributed file-system that stores data on commodity machines, providing very high aggregate bandwidth across the cluster;
- *Hadoop YARN* – a resource-management platform responsible for managing computing resources in clusters and using them for scheduling of users' applications;^{[6][7]} and
- *Hadoop MapReduce* – a programming model for large scale data processing.

The term "Hadoop" has come to refer not just to the base modules above, but also to the "ecosystem",^[8] or collection of additional software packages that can be installed on top of or alongside Hadoop, such as Apache Pig, Apache Hive, Apache HBase, Apache Phoenix, Apache Spark, Apache Zookeeper, Impala, Apache Flume, Apache Sqoop, Apache Oozie, Apache Storm and others.^[9]

Apache Hadoop's MapReduce and HDFS components were inspired by Google papers on their MapReduce and Google File System.^[10]

The Hadoop framework itself is mostly written in the Java programming language, with some native code in C and command line utilities written as Shell script. For end-users, though MapReduce Java code is common, any programming language can be used with "Hadoop Streaming" to implement the "map" and "reduce" parts of the user's program.^[11] Other related projects expose other higher-level user interfaces.

Apache Hadoop



Developer(s)	Apache Software Foundation
Initial release	December 10, 2011 ^[1]
Stable release	2.7.1 / July 6, 2015 ^[2]
Development status	Active
Written in	Java
Operating system	Cross-platform
Type	Distributed file system
License	Apache License 2.0
Website	hadoop.apache.org (http://hadoop.apache.org/)

Contents

- 1 History
- 2 Architecture
 - 2.1 File systems
 - 2.1.1 Hadoop distributed file system
 - 2.1.2 Other file systems
 - 2.2 JobTracker and TaskTracker: the MapReduce engine
 - 2.2.1 Scheduling
 - 2.2.1.1 Fair scheduler
 - 2.2.1.2 Capacity scheduler
 - 2.3 Other applications
- 3 Prominent users
 - 3.1 Yahoo!
 - 3.2 Facebook
 - 3.3 Other users
- 4 Hadoop hosting in the Cloud
 - 4.1 Hadoop on Microsoft Azure
 - 4.2 Hadoop on Amazon EC2/S3 services
 - 4.3 Amazon Elastic MapReduce
- 5 Commercial support
 - 5.1 ASF's view on the use of "Hadoop" in product names
- 6 Papers
- 7 See also
- 8 References
- 9 Bibliography
- 10 External links

History

Hadoop was created by Doug Cutting and Mike Cafarella^[12] in 2005. Cutting, who was working at Yahoo! at the time,^[13] named it after his son's toy elephant.^[14] It was originally developed to support distribution for the Nutch search engine project.^[15]

Architecture

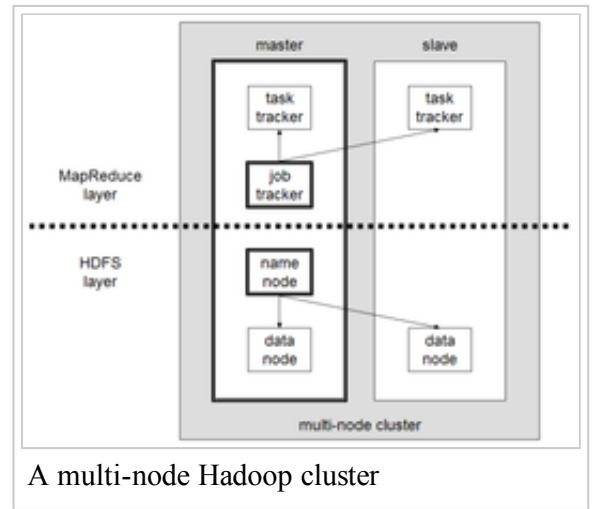
See also: Hadoop Distributed File System, Apache HBase and MapReduce

Hadoop consists of the *Hadoop Common* package, which provides filesystem and OS level abstractions, a MapReduce engine (either MapReduce/MR1 or YARN/MR2)^[16] and the Hadoop Distributed File System (HDFS). The Hadoop Common package contains the necessary Java ARchive (JAR) files and scripts needed to start Hadoop.

For effective scheduling of work, every Hadoop-compatible file system should provide location awareness: the name of the rack (more precisely, of the network switch) where a worker node is. Hadoop applications can use this information to run work on the node where the data is, and, failing that, on the same rack/switch, reducing backbone traffic. HDFS uses this method when replicating data to try to keep different copies of the data on different racks. The goal is to reduce the impact of a rack power outage or switch failure, so that even if these events occur, the data may still be readable.^[17]

A small Hadoop cluster includes a single master and multiple worker nodes. The master node consists of a Job Tracker, Task Tracker, NameNode, and DataNode. A slave or *worker node* acts as both a DataNode and TaskTracker, though it is possible to have data-only worker nodes and compute-only worker nodes. These are normally used only in nonstandard applications.^[18]

Hadoop requires Java Runtime Environment (JRE) 1.6 or higher. The standard startup and shutdown scripts require that Secure Shell (ssh) be set up between nodes in the cluster.^[19]



In a larger cluster, the HDFS is managed through a dedicated NameNode server to host the file system index, and a secondary NameNode that can generate snapshots of the namenode's memory structures, thus preventing file-system corruption and reducing loss of data. Similarly, a standalone JobTracker server can manage job scheduling. In clusters where the Hadoop MapReduce engine is deployed against an alternate file system, the NameNode, secondary NameNode, and DataNode architecture of HDFS are replaced by the file-system-specific equivalents.

File systems

Hadoop distributed file system

The **Hadoop distributed file system (HDFS)** is a distributed, scalable, and portable file-system written in Java for the Hadoop framework. A Hadoop cluster has nominally a single namenode plus a cluster of datanodes, although redundancy options are available for the namenode due to its criticality. Each datanode serves up blocks of data over the network using a block protocol specific to HDFS. The file system uses TCP/IP sockets for communication. Clients use remote procedure call (RPC) to communicate between each other.

HDFS stores large files (typically in the range of gigabytes to terabytes^[20]) across multiple machines. It achieves reliability by replicating the data across multiple hosts, and hence theoretically does not require RAID storage on hosts (but to increase I/O performance some RAID configurations are still useful). With the default replication value, 3, data is stored on three nodes: two on the same rack, and one on a different rack. Data nodes can talk to each other to rebalance data, to move copies around, and to keep the replication of data high. HDFS is not fully POSIX-compliant, because the requirements for a POSIX file-system differ from the target goals for a Hadoop application. The trade-off of not having a fully POSIX-compliant file-system is increased performance for data throughput and support for non-POSIX operations such as Append.^[21]

HDFS added the high-availability capabilities, as announced for release 2.0 in May 2012,^[22] letting the main metadata server (the NameNode) fail over manually to a backup. The project has also started developing automatic fail-over.

The HDFS file system includes a so-called *secondary namenode*, a misleading name that some might incorrectly interpret as a backup namenode for when the primary namenode goes offline. In fact, the secondary namenode regularly connects with the primary namenode and builds snapshots of the primary namenode's directory information, which the system then saves to local or remote directories. These checkpointed images can be used to restart a failed primary namenode without having to replay the

entire journal of file-system actions, then to edit the log to create an up-to-date directory structure. Because the namenode is the single point for storage and management of metadata, it can become a bottleneck for supporting a huge number of files, especially a large number of small files. HDFS Federation, a new addition, aims to tackle this problem to a certain extent by allowing multiple namespaces served by separate namenodes.

An advantage of using HDFS is data awareness between the job tracker and task tracker. The job tracker schedules map or reduce jobs to task trackers with an awareness of the data location. For example: if node A contains data (x,y,z) and node B contains data (a,b,c), the job tracker schedules node B to perform map or reduce tasks on (a,b,c) and node A would be scheduled to perform map or reduce tasks on (x,y,z). This reduces the amount of traffic that goes over the network and prevents unnecessary data transfer. When Hadoop is used with other file systems, this advantage is not always available. This can have a significant impact on job-completion times, which has been demonstrated when running data-intensive jobs.^[23]

HDFS was designed for mostly immutable files^[21] and may not be suitable for systems requiring concurrent write-operations.

HDFS can be mounted directly with a Filesystem in Userspace (FUSE) virtual file system on Linux and some other Unix systems.

File access can be achieved through the native Java API, the Thrift API to generate a client in the language of the users' choosing (C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, Cocoa, Smalltalk, and OCaml), the command-line interface, browsed through the HDFS-UI webapp over HTTP, or via 3rd-party network client libraries.^[24]

Other file systems

Hadoop works directly with any distributed file system that can be mounted by the underlying operating system simply by using a `file://` URL; however, this comes at a price: the loss of locality. To reduce network traffic, Hadoop needs to know which servers are closest to the data; this is information that Hadoop-specific file system bridges can provide.

In May 2011, the list of supported file systems bundled with Apache Hadoop were:

- HDFS: Hadoop's own rack-aware file system.^[25] This is designed to scale to tens of petabytes of storage and runs on top of the file systems of the underlying operating systems.
- FTP File system: this stores all its data on remotely accessible FTP servers.
- Amazon S3 (Simple Storage Service) file system. This is targeted at clusters hosted on the Amazon Elastic Compute Cloud server-on-demand infrastructure. There is no rack-awareness in this file system, as it is all remote.
- Windows Azure Storage Blobs (WASB) (<http://azure.microsoft.com/en-us/documentation/articles/hdinsight-use-blob-storage>) file system. WASB, an extension on top of HDFS, allows distributions of Hadoop to access data in Azure blob stores without moving the data permanently into the cluster.

A number of third-party file system bridges have also been written, none of which are currently in Hadoop distributions. However, some commercial distributions of Hadoop ship with an alternative filesystem as the default—specifically IBM and MapR.

- In 2009, IBM discussed running Hadoop over the IBM General Parallel File System.^[26] The source code was published in October 2009.^[27]

- In April 2010, Parascle published the source code to run Hadoop against the Parascle file system.^[28]
- In April 2010, Appistry released a Hadoop file system driver for use with its own CloudIQ Storage product.^[29]
- In June 2010, HP discussed a location-aware IBRIX Fusion file system driver.^[30]
- In May 2011, MapR Technologies, Inc. announced the availability of an alternative file system for Hadoop, which replaced the HDFS file system with a full random-access read/write file system.

JobTracker and TaskTracker: the MapReduce engine

Main article: MapReduce

Above the file systems comes the MapReduce Engine, which consists of one *JobTracker*, to which client applications submit MapReduce jobs. The JobTracker pushes work out to available *TaskTracker* nodes in the cluster, striving to keep the work as close to the data as possible. With a rack-aware file system, the JobTracker knows which node contains the data, and which other machines are nearby. If the work cannot be hosted on the actual node where the data resides, priority is given to nodes in the same rack. This reduces network traffic on the main backbone network. If a TaskTracker fails or times out, that part of the job is rescheduled. The TaskTracker on each node spawns a separate Java Virtual Machine process to prevent the TaskTracker itself from failing if the running job crashes its JVM. A heartbeat is sent from the TaskTracker to the JobTracker every few minutes to check its status. The Job Tracker and TaskTracker status and information is exposed by Jetty and can be viewed from a web browser.

Known limitations of this approach are:

- The allocation of work to TaskTrackers is very simple. Every TaskTracker has a number of available *slots* (such as "4 slots"). Every active map or reduce task takes up one slot. The Job Tracker allocates work to the tracker nearest to the data with an available slot. There is no consideration of the current system load of the allocated machine, and hence its actual availability.
- If one TaskTracker is very slow, it can delay the entire MapReduce job—especially towards the end of a job, where everything can end up waiting for the slowest task. With speculative execution enabled, however, a single task can be executed on multiple slave nodes.

Scheduling

By default Hadoop uses FIFO scheduling, and optionally 5 scheduling priorities to schedule jobs from a work queue.^[31] In version 0.19 the job scheduler was refactored out of the JobTracker, while adding the ability to use an alternate scheduler (such as the *Fair scheduler* or the *Capacity scheduler*, described next).^[32]

Fair scheduler

The fair scheduler was developed by Facebook.^[33] The goal of the fair scheduler is to provide fast response times for small jobs and QoS for production jobs. The fair scheduler has three basic concepts.^[34]

1. Jobs are grouped into pools.
2. Each pool is assigned a guaranteed minimum share.
3. Excess capacity is split between jobs.

By default, jobs that are uncategorized go into a default pool. Pools have to specify the minimum number of map slots, reduce slots, and a limit on the number of running jobs.

Capacity scheduler

The capacity scheduler was developed by Yahoo. The capacity scheduler supports several features that are similar to the fair scheduler.^[35]

- Queues are allocated a fraction of the total resource capacity.
- Free resources are allocated to queues beyond their total capacity.
- Within a queue a job with a high level of priority has access to the queue's resources.

There is no preemption once a job is running.

Other applications

The HDFS file system is not restricted to MapReduce jobs. It can be used for other applications, many of which are under development at Apache. The list includes the HBase database, the Apache Mahout machine learning system, and the Apache Hive Data Warehouse system. Hadoop can in theory be used for any sort of work that is batch-oriented rather than real-time, is very data-intensive, and benefits from parallel processing of data. It can also be used to complement a real-time system, such as lambda architecture.

As of October 2009, commercial applications of Hadoop^[36] included:

- Log and/or clickstream analysis of various kinds
- Marketing analytics
- Machine learning and/or sophisticated data mining
- Image processing
- Processing of XML messages
- Web crawling and/or text processing
- General archiving, including of relational/tabular data, e.g. for compliance

Prominent users

Yahoo!

On February 19, 2008, Yahoo! Inc. launched what it claimed was the world's largest Hadoop production application. The Yahoo! Search Webmap is a Hadoop application that runs on a Linux cluster with more than 10,000 cores and produced data that was used in every Yahoo! web search query.^[37]

There are multiple Hadoop clusters at Yahoo! and no HDFS file systems or MapReduce jobs are split across multiple datacenters. Every Hadoop cluster node bootstraps the Linux image, including the Hadoop distribution. Work that the clusters perform is known to include the index calculations for the Yahoo! search engine.

On June 10, 2009, Yahoo! made the source code of the version of Hadoop it runs in production available to the public.^[38] Yahoo! contributes all the work it does on Hadoop to the open-source community. The company's developers also fix bugs, provide stability improvements internally, and release this patched source code so that other users may benefit from their effort.

Facebook

In 2010, Facebook claimed that they had the largest Hadoop cluster in the world with 21 PB of storage.^[39] On June 13, 2012, they announced the data had grown to 100 PB.^[40] On November 8, 2012, they announced the data gathered in the warehouse grows by roughly half a PB per day.^[41]

Other users

As of 2013, Hadoop adoption is widespread. For example, more than half of the Fortune 50 use Hadoop.^[42]

Hadoop hosting in the Cloud

Hadoop can be deployed in a traditional onsite datacenter as well as in the cloud.^[43] The cloud allows organizations to deploy Hadoop without hardware to acquire or specific setup expertise.^[44] Vendors who currently have an offer for the cloud include Microsoft, Amazon, and Google.

Hadoop on Microsoft Azure

Azure HDInsight^[45] is a service that deploys Hadoop on Microsoft Azure. HDInsight uses a Windows-based Hadoop distribution that was jointly developed with Hortonworks and allows programming extensions with .NET (in addition to Java). HDInsight also supports creation of Hadoop clusters using Linux with Ubuntu.^[45] By deploying HDInsight in the cloud, organizations can spin up the number of nodes they want and only get charged for the compute and storage that is used.^[45] Hortonworks implementations can also move data from the on-premises datacenter to the cloud for backup, development/test, and bursting scenarios.^[45] It is also possible to run Cloudera or Hortonworks Hadoop clusters on Azure Virtual Machines.

Hadoop on Amazon EC2/S3 services

It is possible to run Hadoop on Amazon Elastic Compute Cloud (EC2) and Amazon Simple Storage Service (S3).^[46] As an example, The New York Times used 100 Amazon EC2 instances and a Hadoop application to process 4 TB of raw image TIFF data (stored in S3) into 11 million finished PDFs in the space of 24 hours at a computation cost of about \$240 (not including bandwidth).^[47]

There is support for the S3 object store in the Apache Hadoop releases, though this is below what one expects from a traditional [Posix] FileSystem. Specifically, operations such as Template:Rename() and Template>Delete() on directories are not atomic, and can take time proportional to the number of entries and the amount of data in them.

Amazon Elastic MapReduce

Elastic MapReduce (EMR)^[48] was introduced by Amazon in April 2009. Provisioning of the Hadoop cluster, running and terminating jobs, and handling data transfer between EC2(VM) and S3(Object Storage) are automated by Elastic MapReduce. Apache Hive, which is built on top of Hadoop for providing data warehouse services, is also offered in Elastic MapReduce.^[49]

Support for using Spot Instances^[50] was later added in August 2011.^[51] Elastic MapReduce is fault tolerant for slave failures,^[52] and it is recommended to only run the Task Instance Group on spot instances to take advantage of the lower cost while maintaining availability.^[53]

Commercial support

A number of companies offer commercial implementations or support for Hadoop.^[54]

ASF's view on the use of "Hadoop" in product names

The Apache Software Foundation has stated that only software officially released by the Apache Hadoop Project can be called *Apache Hadoop* or *Distributions of Apache Hadoop*.^[55] The naming of products and derivative works from other vendors and the term "compatible" are somewhat controversial within the Hadoop developer community.^[56]

Papers

Some papers influenced the birth and growth of Hadoop and big data processing. Here is a partial list:

- Jeffrey Dean, Sanjay Ghemawat (2004) MapReduce: Simplified Data Processing on Large Clusters (https://www.usenix.org/legacy/publications/library/proceedings/osdi04/tech/full_papers/dean/dean_html/index.html), Google. This paper inspired Doug Cutting to develop an open-source implementation of the Map-Reduce framework. He named it Hadoop, after his son's toy elephant.
- Michael Franklin, Alon Halevy, David Maier (2005) From Databases to Dataspaces: A New Abstraction for Information Management (<http://www.eecs.berkeley.edu/~franklin/Papers/dataspaceSR.pdf>). The authors highlight the need for storage systems to accept all data formats and to provide APIs for data access that evolve based on the storage system's understanding of the data.
- Fay Chang et al. (2006) Bigtable: A Distributed Storage System for Structured Data (http://static.googleusercontent.com/external_content/untrusted_dlcp/research.google.com/en/us/archive/bigtable-osdi06.pdf), Google.
- Robert Kallman et al. (2008) H-store: a high-performance, distributed main memory transaction processing system (<http://www.vldb.org/pvldb/1/1454211.pdf>)

See also

- Apache Accumulo – Secure BigTable^[57]
- Apache Cassandra – A column-oriented database that supports access from Hadoop
- Apache CouchDB is a database that uses JSON for documents, JavaScript for MapReduce queries, and regular HTTP for an API
- Big data
- Cloud computing
- Data Intensive Computing
- HPCC – LexisNexis Risk Solutions High Performance Computing Cluster
- Hypertable – HBase alternative
- Sector/Sphere – Open source distributed storage and processing
- Simple Linux Utility for Resource Management

References

1. "Hadoop Releases". *apache.org*. Apache Software Foundation. Retrieved 2014-12-06.
2. "Hadoop Releases". Hadoop.apache.org. Retrieved 2015-07-29.
3. "Welcome to Apache™ Hadoop®!". *hadoop.apache.org*. Retrieved 2015-09-20.
4. "What is the Hadoop Distributed File System (HDFS)?". *ibm.com*. IBM. Retrieved 2014-10-30.
5. Malak, Michael (2014-09-19). "Data Locality: HPC vs. Hadoop vs. Spark". *datascienceassn.org*. Data Science Association. Retrieved 2014-10-30.
6. "Resource (Apache Hadoop Main 2.5.1 API)". *apache.org*. Apache Software Foundation. 2014-09-12. Retrieved 2014-09-30.
7. Murthy, Arun (2012-08-15). "Apache Hadoop YARN – Concepts and Applications". *hortonworks.com*. Hortonworks. Retrieved 2014-09-30.
8. "Continuity Raises \$10 Million Series A Round to Ignite Big Data Application Development Within the Hadoop Ecosystem". *finance.yahoo.com*. Marketwired. 2012-11-14. Retrieved 2014-10-30.
9. "Hadoop-related projects at". Hadoop.apache.org. Retrieved 2013-10-17.
10. *Data Science and Big Data Analytics: Discovering, Analyzing, Visualizing and Presenting Data*. John Wiley & Sons. 2014-12-19. p. 300. ISBN 9781118876220. Retrieved 2015-01-29.
11. "[nlpatumd] Adventures with Hadoop and Perl". Mail-archive.com. 2010-05-02. Retrieved 2013-04-05.
12. "Michael J. Cafarella". Web.eecs.umich.edu. Retrieved 2013-04-05.
13. Hadoop creator goes to Cloudera (<http://www.sdtimes.com/blog/post/2009/08/10/Hadoop-creator-goes-to-Cloudera.aspx>)
14. Vance, Ashlee (2009-03-17). "Hadoop, a Free Software Program, Finds Uses Beyond Search". *The New York Times*. Archived from the original on 11 February 2010. Retrieved 2010-01-20.
15. "Hadoop contains the distributed computing platform that was formerly a part of Nutch. This includes the Hadoop Distributed Filesystem (HDFS) and an implementation of MapReduce." About Hadoop (<http://hadoop.apache.org/core/>)
16. Chouraria, Harsh (21 October 2012). "MR2 and YARN Briefly Explained". *cloudera.com*. Cloudera. Retrieved 23 October 2013.
17. "HDFS User Guide". Hadoop.apache.org. Retrieved 2014-09-04.
18. "Running Hadoop on Ubuntu Linux System(Multi-Node Cluster)".
19. "Running Hadoop on Ubuntu Linux (Single-Node Cluster)". Retrieved 6 June 2013.
20. "HDFS Architecture". Retrieved 1 September 2013.
21. Pessach, Yaniv (2013). "Distributed Storage" (Distributed Storage: Concepts, Algorithms, and Implementations ed.). Amazon.com
22. "Version 2.0 provides for manual failover and they are working on automatic failover:". Hadoop.apache.org. Retrieved 30 July 2013.
23. "Improving MapReduce performance through data placement in heterogeneous Hadoop Clusters" (PDF). Eng.auburn.ed. April 2010.
24. "Mounting HDFS". Retrieved May 2014.
25. "HDFS Users Guide – Rack Awareness". Hadoop.apache.org. Retrieved 2013-10-17.
26. "Cloud analytics: Do we really need to reinvent the storage stack?" (PDF). IBM. June 2009.
27. "HADOOP-6330: Integrating IBM General Parallel File System implementation of Hadoop Filesystem interface". IBM. 2009-10-23.
28. "HADOOP-6704: add support for Parascale filesystem". Parascale. 2010-04-14.
29. "HDFS with CloudIQ Storage". Appistry,Inc. 2010-07-06.
30. "High Availability Hadoop". HP. 2010-06-09.
31. job (http://hadoop.apache.org/common/docs/current/commands_manual.html#job)
32. "Refactor the scheduler out of the JobTracker". *Hadoop Common*. Apache Software Foundation. Retrieved 9 June 2012.
33. Jones, M. Tim (6 December 2011). "Scheduling in Hadoop". *ibm.com*. IBM. Retrieved 20 November 2013.
34. Hadoop Fair Scheduler Design Document (https://svn.apache.org/repos/asf/hadoop/common/branches/MAPREDUCE-233/src/contrib/fairscheduler/designdoc/fair_scheduler_design_doc.pdf)
35. Capacity Scheduler Guide (http://hadoop.apache.org/docs/stable1/capacity_scheduler.html)
36. "'How 30+ enterprises are using Hadoop', in DBMS2". Dbms2.com. 10 October 2009. Retrieved 2013-10-17.

37. Yahoo! Launches World's Largest Hadoop Production Application (<https://developer.yahoo.com/blogs/hadoop/yahoo-launches-world-largest-hadoop-production-application-398.html>)
38. "Hadoop and Distributed Computing at Yahoo!". Yahoo!. 2011-04-20. Retrieved 2013-10-17.
39. "HDFS: Facebook has the world's largest Hadoop cluster!". Hadoopblog.blogspot.com. 2010-05-09. Retrieved 2012-05-23.
40. "Under the Hood: Hadoop Distributed File system reliability with Namenode and Avatarnode". Facebook. Retrieved 2012-09-13.
41. "Under the Hood: Scheduling MapReduce jobs more efficiently with Corona". Facebook. Retrieved 2012-11-09.
42. "Altior's AltraSTAR – Hadoop Storage Accelerator and Optimizer Now Certified on CDH4 (Cloudera's Distribution Including Apache Hadoop Version 4)" (Press release). Eatontown, NJ: Altior Inc. 2012-12-18. Retrieved 2013-10-30.
43. "What is Hadoop?".
44. "Hadoop". Azure.microsoft.com. Retrieved 2014-07-22.
45. "HDInsight | Cloud Hadoop". Azure.microsoft.com. Retrieved 2014-07-22.
46. Varia, Jinesh (@jinman). "Taking Massive Distributed Computing to the Common Man – Hadoop on Amazon EC2/S3". *Amazon Web Services Blog*. Amazon.com. Retrieved 9 June 2012.
47. Gottfrid, Derek (1 November 2007). "Self-service, Prorated Super Computing Fun!". *The New York Times*. Retrieved 4 May 2010.
48. "AWS | Amazon Elastic MapReduce (EMR) | Hadoop MapReduce in the Cloud". Aws.amazon.com. Retrieved 2014-07-22.
49. "Amazon Elastic MapReduce Developer Guide" (PDF). Retrieved 2013-10-17.
50. "Amazon EC2 Spot Instances". Aws.amazon.com. Retrieved 2014-07-22.
51. "Amazon Elastic MapReduce Now Supports Spot Instances". Amazon.com. 2011-08-18. Retrieved 2013-10-17.
52. "Amazon Elastic MapReduce FAQs". Amazon.com. Retrieved 2013-10-17.
53. Using Spot Instances with EMR (<https://www.youtube.com/watch?v=66rfnFA0jpM>) on YouTube
54. "Why the Pace of Hadoop Innovation Has to Pick Up". Gigaom.com. 2011-04-25. Retrieved 2013-10-17.
55. "Defining Hadoop". Wiki.apache.org. 2013-03-30. Retrieved 2013-10-17.
56. "Defining Hadoop Compatibility: revisited". Mail-archives.apache.org. 2011-05-10. Retrieved 2013-10-17.
57. "Apache Accumulo User Manual: Security". *apache.org*. Apache Software Foundation. Retrieved 2014-12-03.

Bibliography

- Lam, Chuck (July 28, 2010). *Hadoop in Action* (1st ed.). Manning Publications. p. 325. ISBN 1-935-18219-6.
- Venner, Jason (June 22, 2009). *Pro Hadoop* (1st ed.). Apress. p. 440. ISBN 1-430-21942-4.
- White, Tom (June 16, 2009). *Hadoop: The Definitive Guide* (1st ed.). O'Reilly Media. p. 524. ISBN 0-596-52197-9.

External links

- Official Hadoop Homepage (<http://hadoop.apache.org>)
- Apache Hadoop popular APIs in GitHub (<http://apiwave.com/java/api/org.apache.hadoop>)
- Introducing Apache Hadoop: The Modern Data Operating System (<http://www.stanford.edu/class/ee380/Abstracts/111116.html>) — lecture given at Stanford University by Co-Founder and CTO of Cloudera, Amr Awadallah (video archive (<http://ee380.stanford.edu/cgi-bin/videologger.php?target=111116-ee380-300.asx>)) (YouTube (<http://www.youtube.com/watch?v=d2xeNpfzsYI>)))
- Hadoop with Philip Zeyliger, Software Engineering Radio, IEEE Computer Society, March 8 2010 (<http://www.se-radio.net/2010/03/episode-157-hadoop-with-philip-zeyliger/>)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Apache_Hadoop&oldid=689128635"

Categories: Hadoop | Apache Software Foundation | Software using the Apache license
| Free software programmed in Java (programming language) | Free system software
| Distributed file systems | Cloud infrastructure | Free software for cloud computing

- This page was last modified on 5 November 2015, at 03:03.
- Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.